

Pixel To Geometry (PTG)

Chad B. Hovey¹

Sandia Injury Biomechanics Laboratory
Sandia National Laboratories²

2020-04-10

¹Corresponding author: chovey@sandia.gov

²The Sandia National Laboratories is a multimission laboratory managed and operated by National Technology and Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International, Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

Abstract

Classification of pixel values from human CT/MR scans has been successfully undertaken by this laboratory using the U-Net deep learning algorithm. We have also demonstrated successful conversion of pixels to voxels, as a proxy for finite volume or finite element meshes, a process we have called Pixel To Mesh (PTM). This mesh step quickly becomes intractable when the number of voxels grows past the 10s of millions. Essentially, a map of every pixel (voxel) to a single finite volume or finite element creates computational domains of excessively large and equally unnecessary mesh refinement.

In light of this, we investigated an alternative approach, which we call Pixel To Geometry (PTG). This alternative seeks to selectively use the categorized pixel values as a scaffold for construction of analytical geometric descriptions in 2D and 3D. Thereafter, we propose using the analytical geometries directly in simulation, using isogeometric analysis (IGA). The use of IGA completely eliminates any need to complete the mesh step.

Contents

1	Introduction	3
2	Methods	4
2.1	Non-local Methods	4
2.1.1	Polynomials	4
2.1.2	Bézier Curves	5
2.2	Local Methods	11
2.2.1	General Splines	11
2.2.2	Linear Splines	11
2.2.3	Quadratic Splines	13
2.2.4	Cubic Splines	13

Chapter 1

Introduction

The concept of converting pixels to a geometric representation, hereinafter referred to as PTG (Pixel To Geometry), can be introduced by consideration of two similar processes, optical character recognition (OCR) and analytic geometry.

OCR is the process of converting pixel values scanned from a piece of paper with printed or handwritten text into a digital encoding of letters and number, e.g.,

A a B b C c ... Z z 0 1 2 3 4 5 6 7 8 9, where a collection of pixels is composed to map to letters and numbers.

Figure 1.1 illustrates the difference between vector graphics and rasterized graphics.

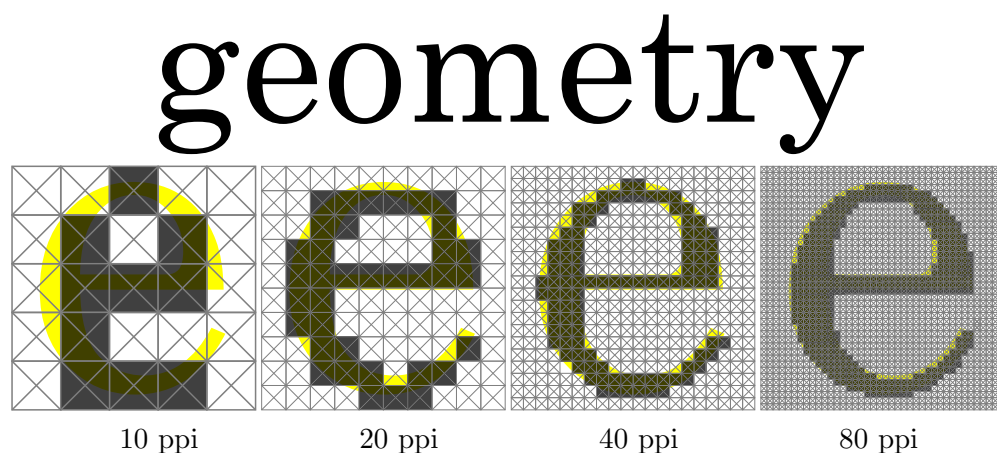


Figure 1.1: The word “geometry” appearing with font size 48 point (72 points = 1 inch) composed of vector graphics, and the “e” in yellow, with font size 72 point, appearing from left-to-right at half-inch intervals in rasterized graphics at 10 pixels per inch (ppi), 20 ppi, 40 ppi, and 80 ppi. The image was saved in vectorized format at 600 ppi. Reference: fig/geometry.pdf

Chapter 2

Methods

2.1 Non-local Methods

2.1.1 Polynomials

Let $f(x)$ be the function of order p that is a sum of variable x raised to some increasing non-negative integer power, starting from zero, and multiplied by a constant coefficient a , such that

$$f(x) \triangleq a_0 + a_1x + a_2x^2 + a_3x^3 + \cdots + a_px^p = \sum_{i=0}^p a_ix^i. \quad (2.1)$$

The function $f(x)$ is a **polynomial** of **degree** p , the highest non-zero power used in the summation. Table 2.1 lists the first five polynomials by degree, name, and function.

Table 2.1: First five orders of a polynomial function.

degree	name	function
$p = 0$	constant	$f_0(x) = a_0$
$p = 1$	linear	$f_1(x) = a_0 + a_1x$
$p = 2$	quadratic	$f_2(x) = a_0 + a_1x + a_2x^2$
$p = 3$	cubic	$f_3(x) = a_0 + a_1x + a_2x^2 + a_3x^3$
$p = 4$	quartic	$f_4(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4$

For a polynomial of degree p , the function has $p - 1$ changes of direction, from $-f(x)$ to $f(x)$, or vice versa. This characteristic leads causes an overfitting behavior, which makes polynomials disadvantageous to use. For this reason, polynomials are not further explicated. They are simply included here for context.

2.1.2 Bézier Curves

Several of the ideas in this section are due to the excellent references of [Bartels 1995], [Rogers 2000], and [Shiach 2015]. A Bézier curve is a parametric curve defined by control points. A control point $\mathbf{P}_i$ will have two coordinates (x_i, y_i) in 2D and three coordinates (x_i, y_i, z_i) in 3D. The parameter is typically denoted t . The bounds for t are always $0 \leq t \leq 1$.

A Bézier curve of order p requires $p + 1$ control points. For example, a Bézier curve that is a line (order $p = 1$) requires two control points. Affine transformations may be used to modify the control points, e.g., to scale, reflect, rotate, or translate (offset) the control points.

Bézier Line

Let $\mathbf{P}(t; \mathbf{x})$ be **parametric** equation for a line between two points $\mathbf{P}_0 = (x_0, y_0, z_0)$ and $\mathbf{P}_1 = (x_1, y_1, z_1)$, with $\mathbf{x} = (x, y, z)$. Then, let t be the parameter bounded by $0 \leq t \leq 1$. Then the Bézier line between $\mathbf{x}_0$ and $\mathbf{x}_1$ is simply the parameterized linear interpolation $\mathbf{P}_0$ and $\mathbf{P}_1$,

$$\mathbf{P}(t; \mathbf{P}_0, \mathbf{P}_1) \triangleq (1 - t)\mathbf{P}_0 + t\mathbf{P}_1, \quad (2.2)$$

or in explicit coordinate form,

$$\begin{Bmatrix} x(t) \\ y(t) \\ z(t) \end{Bmatrix} = (1 - t) \begin{Bmatrix} x_0 \\ y_0 \\ z_0 \end{Bmatrix} + t \begin{Bmatrix} x_1 \\ y_1 \\ z_1 \end{Bmatrix}. \quad (2.3)$$

Bézier Quadratic

Let $\mathbf{Q}(t; \mathbf{x})$ be the parametric equation for a quadratic between two points $\mathbf{Q}_0$ and $\mathbf{Q}_1$. Let the position of these two points themselves be parameterized by three points $\mathbf{P}_0$, $\mathbf{P}_1$, and $\mathbf{P}_2$, such that

$$\mathbf{Q}_0(t; \mathbf{P}_0, \mathbf{P}_1) = (1 - t)\mathbf{P}_0 + t\mathbf{P}_1, \quad (2.4)$$

$$\mathbf{Q}_1(t; \mathbf{P}_1, \mathbf{P}_2) = (1 - t)\mathbf{P}_1 + t\mathbf{P}_2. \quad (2.5)$$

Thus, the position of $\mathbf{Q}_0$ is on the line between points $\mathbf{P}_0$ and $\mathbf{P}_1$. The position of $\mathbf{Q}_1$ is on the line between points $\mathbf{P}_1$ and $\mathbf{P}_2$. At $t = 0$, $\mathbf{Q}_0$ resides at $\mathbf{P}_0$, $\mathbf{Q}_1$ resides at $\mathbf{P}_1$. At $t = 1$, $\mathbf{Q}_0$ resides at $\mathbf{P}_1$, $\mathbf{Q}_1$ resides at $\mathbf{P}_2$. Finally, let position along the quadratic Bézier curve $\mathbf{Q}(t; \mathbf{x})$ be defined as

$$\mathbf{Q}(t; \mathbf{Q}_0, \mathbf{Q}_1) \triangleq (1 - t)\mathbf{Q}_0 + t\mathbf{Q}_1. \quad (2.6)$$

This curve can be recast in terms of $\mathbf{P}$ points using (2.4) and (2.5), producing

$$\mathbf{Q}(t; \mathbf{P}_0, \mathbf{P}_1, \mathbf{P}_2) = (1 - t)^2\mathbf{P}_0 + 2t(1 - t)\mathbf{P}_1 + t^2\mathbf{P}_2. \quad (2.7)$$

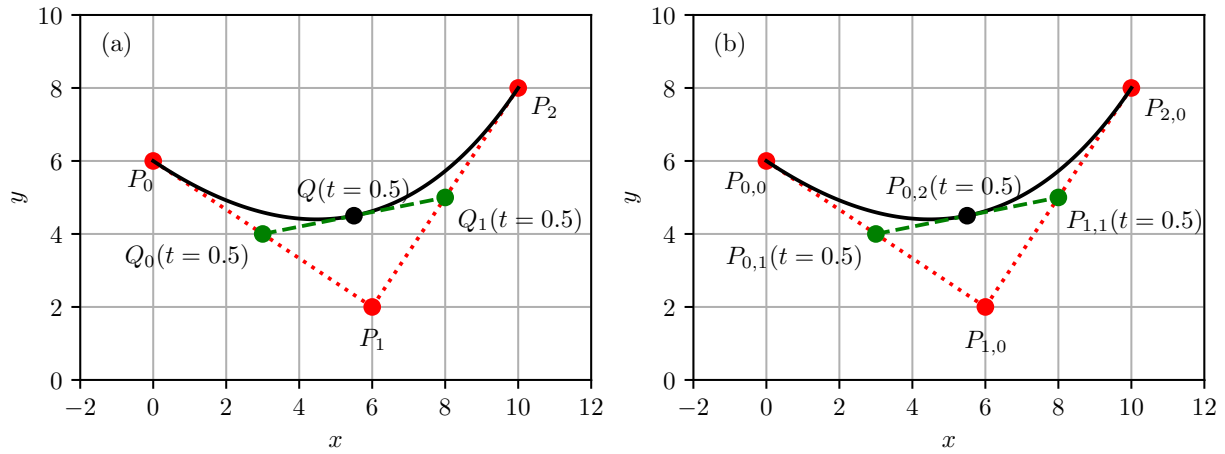


Figure 2.1: A Bézier quadratic curve illustrated at $t = 0.5$ with (a) original notation and (b) the de Casteljau's algorithm notation. The number of control points is three, so the maximum order Bézier that can be constructed is two (order p requires $p + 1$ control points). Reference: fig/de.casteljau.py.

Figure 2.1 illustrates an example quadratic Bézier curve, with control points indicated.

Example 1.

Figure 2.2 illustrates the same quadratic Bézier curve, generated at six discrete points in the interval $t \in [0, 1]$. $\square$

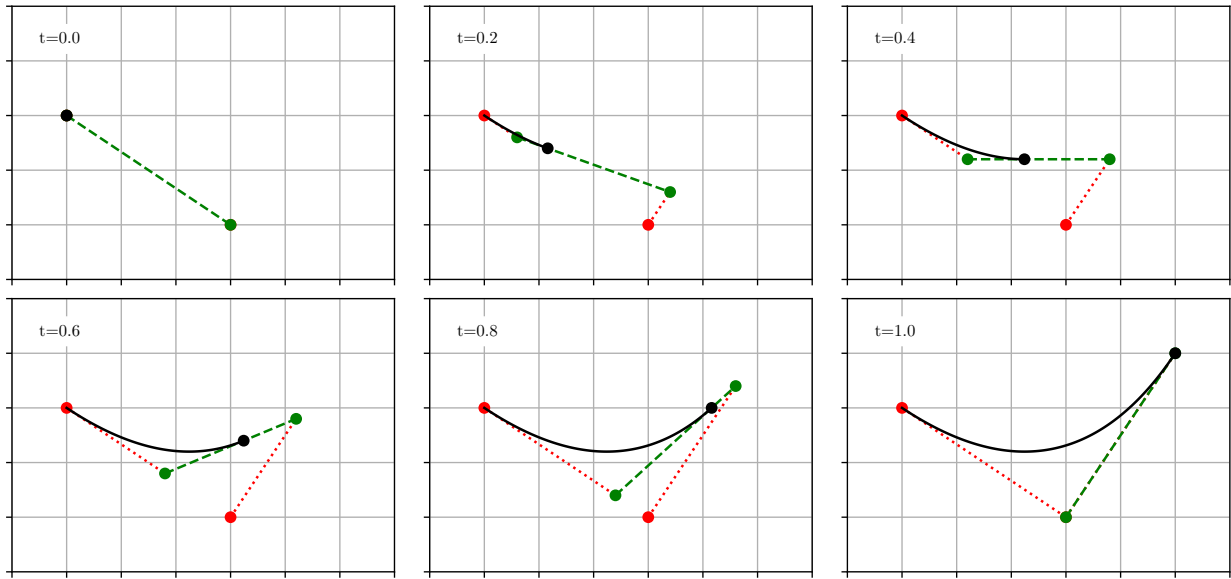


Figure 2.2: The Bézier quadratic curve discussed in Figure 2.1, illustrated in sequence with t starting at 0 and ending at 1. Reference: `fig/de_casteljau.py`.

de Casteljau's Algorithm

The foregoing development can be generalized, and is known as **de Casteljau's algorithm**. Let $P_{i,j}$ be control points where

- i denotes the **control point index**, $i = 0, 1, \dots, n$, (total of $n + 1$ control points), and
- j denote the **order** of the curve, $j = 0, 1, \dots, p$.

Thus $P_{i,0}$ denote the base level control points. In the preceding section,

$$P_0 \mapsto P_{0,0}, \quad j = 0, \text{ thus } P_{i,0} \text{ is a point,} \quad (2.8)$$

$$P_1 \mapsto P_{1,0}, \quad (2.9)$$

$$P_2 \mapsto P_{2,0}, \quad (2.10)$$

$$Q_0 \mapsto P_{0,1}, \quad j = 1, \text{ thus } P_{i,1} \text{ is a line,} \quad (2.11)$$

$$Q_1 \mapsto P_{1,1}, \quad \text{and} \quad (2.12)$$

$$Q(t) \mapsto P_{0,2}, \quad j = 2, \text{ thus } P_{0,2} \text{ is a quadratic.} \quad (2.13)$$

Then, **de Casteljau's algorithm** states

$$\mathbf{P}_{i,j} = (1-t)\mathbf{P}_{i,j-1} + t\mathbf{P}_{i+1,j-1}. \quad (2.14)$$

Equation (2.7) would then be rewritten as

$$\mathbf{P}_{0,2}(t) = (1-t)\mathbf{P}_{0,1} + t\mathbf{P}_{1,1}, \quad (2.15)$$

$$= (1-t)[(1-t)\mathbf{P}_{0,0} + t\mathbf{P}_{1,0}] + t[(1-t)\mathbf{P}_{1,0} + t\mathbf{P}_{2,0}], \quad (2.16)$$

$$= (1-t)^2\mathbf{P}_{0,0} + 2t(1-t)\mathbf{P}_{1,0} + t^2\mathbf{P}_{2,0}, \quad (2.17)$$

where only the t parameter is used to describe $\mathbf{P}_{0,2}(\cdot)$ for the sake of brevity. Figure 2.1 illustrates for the Bézier quadratic development, (a) the original notation and (b) the de Casteljau's algorithm notation.

Bézier Cubic

The cubic Bézier curve $\mathbf{P}_{0,3}(t)$, given four control points $\mathbf{P}_{0,0}$, $\mathbf{P}_{1,0}$, $\mathbf{P}_{2,0}$, and $\mathbf{P}_{3,0}$, is given by

$$\mathbf{P}_{0,3}(t) = (1-t)^3\mathbf{P}_{0,0} + 3t(1-t)^2\mathbf{P}_{1,0} + 3t^2(1-t)\mathbf{P}_{2,0} + t^3\mathbf{P}_{3,0}. \quad (2.18)$$

Bernstein Polynomials

The general form of a degree p Bézier curve $\mathbf{C}(t)$ defined by $p+1$ control points $\mathbf{P}_i$, $i = 0, 1, \dots, p$, is given by

$$\mathbf{C}(t) = \sum_{i=0}^p b_{i,p}(t)\mathbf{P}_i, \quad (2.19)$$

where $b_{i,p}(t)$ is a **Bernstein polynomial**, defined as

$$b_{i,p}(t) = \binom{p}{i} t^i (1-t)^{p-i}. \quad (2.20)$$

Here,

$$\binom{p}{i} = \frac{p!}{i!(p-i)!} \quad (2.21)$$

is the **binomial coefficient**. The binomial coefficients are readily attained from **Pascal's triangle**, written up to $p = 4$ in Table 2.2.

Table 2.2: First five orders of binomial coefficients from Pascal's triangle.

degree	binomial coefficient				
$p = 0$	1				
$p = 1$		1		1	
$p = 2$		1	2	1	
$p = 3$		1	3	3	1
$p = 4$	1	4	6	4	1

Example 2.

Find the Bernstein polynomials for a cubic ($p = 3$) Bézier curve. These are

$$b_{0,3}(t) = \binom{3}{0} t^0 (1-t)^{3-0} = (1-t)^3, \quad (2.22)$$

$$b_{1,3}(t) = \binom{3}{1} t^1 (1-t)^{3-1} = 3t(1-t)^2, \quad (2.23)$$

$$b_{2,3}(t) = \binom{3}{2} t^2 (1-t)^{3-2} = 3t^2(1-t), \quad (2.24)$$

$$b_{3,3}(t) = \binom{3}{3} t^3 (1-t)^{3-3} = t^3, \quad (2.25)$$

which match the coefficients in (2.18). $\square$

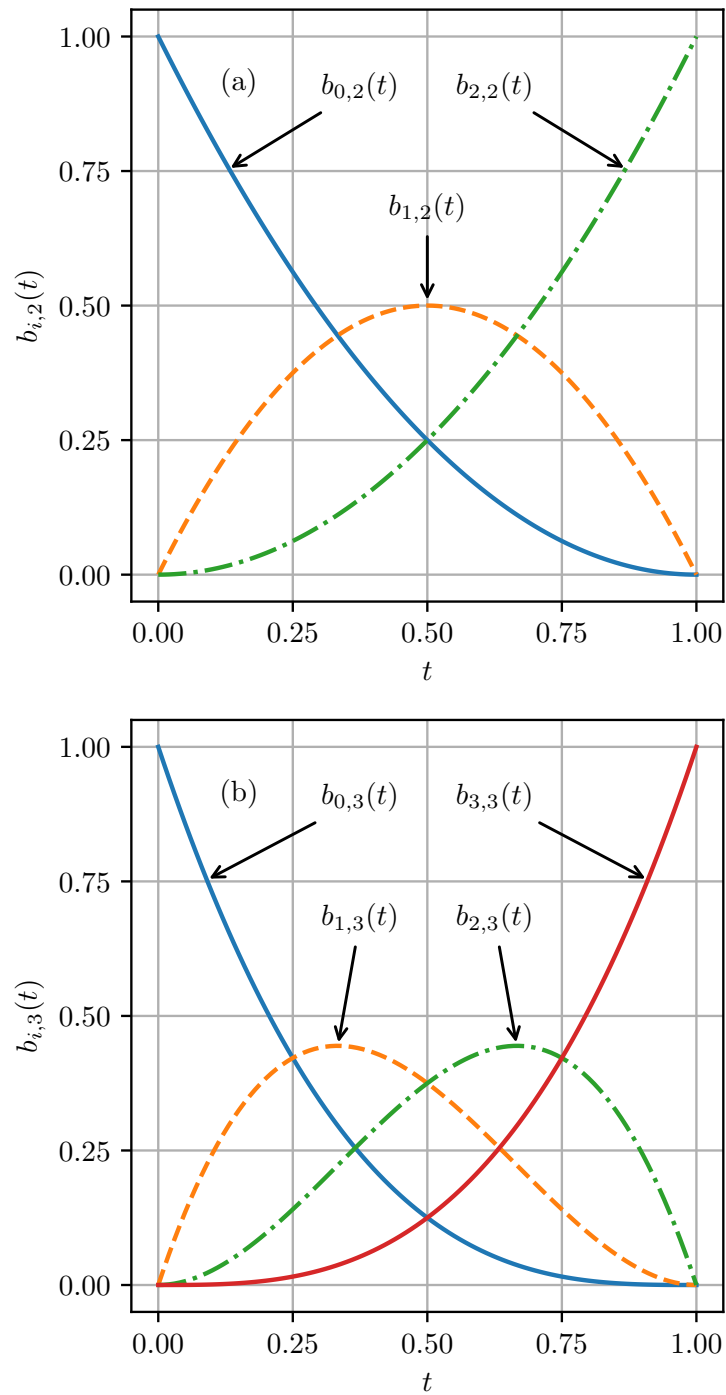


Figure 2.3: Bernstein polynomials for (a) quadratic Bézier ($p = 2$), and (b) cubic Bézier ($p = 3$).
Reference: fig/bernstein.py.

2.2 Local Methods

2.2.1 General Splines

Let a collection of $n+1$ points (x_i, y_i) for $i = 0, 1, \dots, n$ approximate the function $y = f(x)$ over the domain $[x_0, x_n]$. In total, there are $n+1$ points and n intervals. Let $S(x)$ be the piecewise continuous spine that threads through each of the points (x_i, y_i) . For $x \in [x_i, x_{i+1}]$ and $i = 0, 1, \dots, n-1$, the spline takes on n interval definitions as follows:

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 + \dots + z_i(x - x_i)^p. \quad (2.26)$$

where constant coefficients¹ multiply successive powers² of $(x - x_i)$, summed to an arbitrary polynomial power p . This is referred to as the **interval spline definition** to polynomial order p . Below we consider linear splines ($p = 1$), quadratic splines ($p = 2$), and cubic splines ($p = 3$).

2.2.2 Linear Splines

For linear splines, the interval spline definition is

$$S_i(x) \triangleq a_i + b_i(x - x_i) \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (2.27)$$

For any collection of points (x_i, y_i) , all coefficients a_i and b_i for $i = 0, 1, \dots, n-1$ can be found using the **zeroth derivative continuity constraint**, which creates a spline that connects to each point at the beginning and end of each interval, thus

$$S_i(x_i) = y_i \quad \text{and} \quad S_i(x_{i+1}) = y_{i+1} \quad \text{for } i = 0, 1, \dots, n-1. \quad (2.28)$$

Here we see there are two constraints per interval i and two unknowns, a_i and b_i , per interval i . The unknowns can be found directly by evaluating the interval spline definition with each of the two constraint equations. In turn, these are, for $i = 0, 1, \dots, n-1$,

$$S_i(x_i) = a_i + b_i(x_i - x_i) \stackrel{\text{set}}{=} y_i \implies a_i = y_i, \text{ and} \quad (2.29)$$

$$S_i(x_{i+1}) = a_i + b_i(x_{i+1} - x_i) \stackrel{\text{set}}{=} y_{i+1} \implies b_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}. \quad (2.30)$$

No additional continuity constraints can be created since the linear spline has only C^0 continuity, meaning no derivatives of the function $S_i(x)$ are continuous.

¹Here the z_i is arbitrary, meant to denote the last coefficient, but not necessarily the twenty-sixth coefficient.

²The powers are non-negative integers.

Example 3.

Let $y = f(x) = \sin(\pi x)$ for $x \in [0, 1]$ be approximated by a linear spline and a collection of three points equally spaced on the domain. Note $n + 1 = 3$, thus $n = 2$, indicating three points and two intervals. Table 2.3 summarizes the spline data. Figure 2.4(a) shows the $\sin$ function and the linear spline interpolations. $\square$

Table 2.3: Linear spline approximation of $y = f(x) = \sin(\pi x)$ using three points and two intervals.

i	x_i	y_i	$\implies$	a_i	b_i	$\implies$	$S_i(x)$
0	0/2	0		0	2		$2x$
1	1/2	1		1	-2		$1 - 2(x - 1/2)$
2	2/2	0		—	—		—

Example 4.

Repeat the previous example, but now with a collection of five points equally spaced on the domain. Thus $n + 1 = 5$, thus $n = 4$, indicating five points and four intervals. Table 2.4 summarizes the spline data. Figure 2.4(b) shows the sin function and the linear spline interpolations. $\square$

Table 2.4: Linear spline approximation of $y = f(x) = \sin(\pi x)$ using five points and four intervals.

i	x_i	y_i	$\Rightarrow$	a_i	b_i	$\Rightarrow$	$S_i(x)$
0	0/4	0		0	$2\sqrt{2}$		$2\sqrt{2}x$
1	1/4	$\sqrt{2}/2$		$\sqrt{2}/2$	$4 - 2\sqrt{2}$		$\sqrt{2}/2 + (4 - 2\sqrt{2})(x - 1/4)$
2	2/4	1		1	$2\sqrt{2} - 4$		$1 + (2\sqrt{2} - 4)(x - 2/4)$
3	3/4	$\sqrt{2}/2$		$\sqrt{2}/2$	$-2\sqrt{2}$		$\sqrt{2}/2 - 2\sqrt{2}(x - 3/4)$
4	4/4	0		—	—		—

2.2.3 Quadratic Splines

For quadratic splines, the interval spline definition is

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (2.31)$$

2.2.4 Cubic Splines

Cubic splines are useful because of their inherent smoothness. The cubic spline, its first derivative, and its second derivative are all continuous. Starting with the cubic spine's third derivative, discontinuity arises.

Let a collection of $n + 1$ points (x_i, y_i) for $i = 0, 1, \dots, n$ approximate the function $y = f(x)$ over the domain $[x_0, x_n]$. In total, there are $n + 1$ points and n intervals. Let $S(x)$ be the piecewise continuous spine, cubic in this case, that threads through each of the points (x_i, y_i) . The spline takes on n interval definitions as follows:

$$S_i(x) \triangleq a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad \text{for } x \in [x_i, x_{i+1}] \text{ and } i = 0, 1, \dots, n-1. \quad (2.32)$$

The n foregoing definitions, with the coefficients a_i, b_i, c_i, d_i , give rise to $4n$ (currently unknown) parameters used to define the complete spline $S(x)$. Note that maintain its cubic nature, the spline must have $d_i \neq 0$.

Next we endow the spline with constraints on continuity and differentiability, which allow the $4n$ unknown parameters to be solved. First, function continuity requires that the spline, on a particular interval, thread through collection of points (x_i, y_i) and the beginning and end of each interval. Thus,

$$S_i(x_i) = y_i \text{ and } S_i(x_{i+1}) = y_{i+1} \quad \text{for } i = 0, 1, \dots, n-1. \quad (2.33)$$

This is the **continuity constraint**, which gives rise to $2n$ conditions³ and creates a spline that is piecewise continuous. Next, consider two **differentiability constraints**, which state that the first and second derivatives at a point x_i are continuous. That is, the first and second derivatives approaching a point x_i from the interval to the left side of the point must match the respective first and second derivatives approaching the point x_i from the interval to the right side of the point:

$$S'_{i-1}(x_i) = S'_i(x_i) \text{ and } S''_{i-1}(x_i) = S''_i(x_i) \text{ for } i = 1, 2, \dots, n-1. \quad (2.34)$$

Because they apply only to the interior points (and not the endpoints x_0 and x_n), the differentiability constraints give rise to $2(n-1)$ constraints.⁴

Finally, two more conditions are necessary to completely define the spline $S(x)$. The two conditions are obtained by specifying differentiability conditions on the two spline endpoints x_0 and x_n . Frequently, the first derivative is set to the known value of the function's derivatives at the endpoints,

$$S'_0(x_0) = f'(x_0) \text{ and } S'_{n-1}(x_n) = f'(x_n). \quad (2.35)$$

This is defined as a **clamped spline**. Alternatively, a **natural spline** defines the second derivatives to be zero at the endpoints,

$$S''_0(x_0) = 0 \text{ and } S''_{n-1}(x_n) = 0. \quad (2.36)$$

Finally, a **not-a-knot** spline requires no additional constraints on the endpoint derivatives,⁵ but requires that the *third* derivative be continuous at x_1 and x_{n-1} .

³Two conditions per interval times n intervals yields $2n$ conditions.

⁴Two conditions per interior point times $n-1$ interior points yields $2(n-1)$ conditions.

⁵This has no effect on the original continuity constraint on the endpoints, that $S_0(x_0) = y_0$ and $S_{n-1}(x_n) = y_n$.

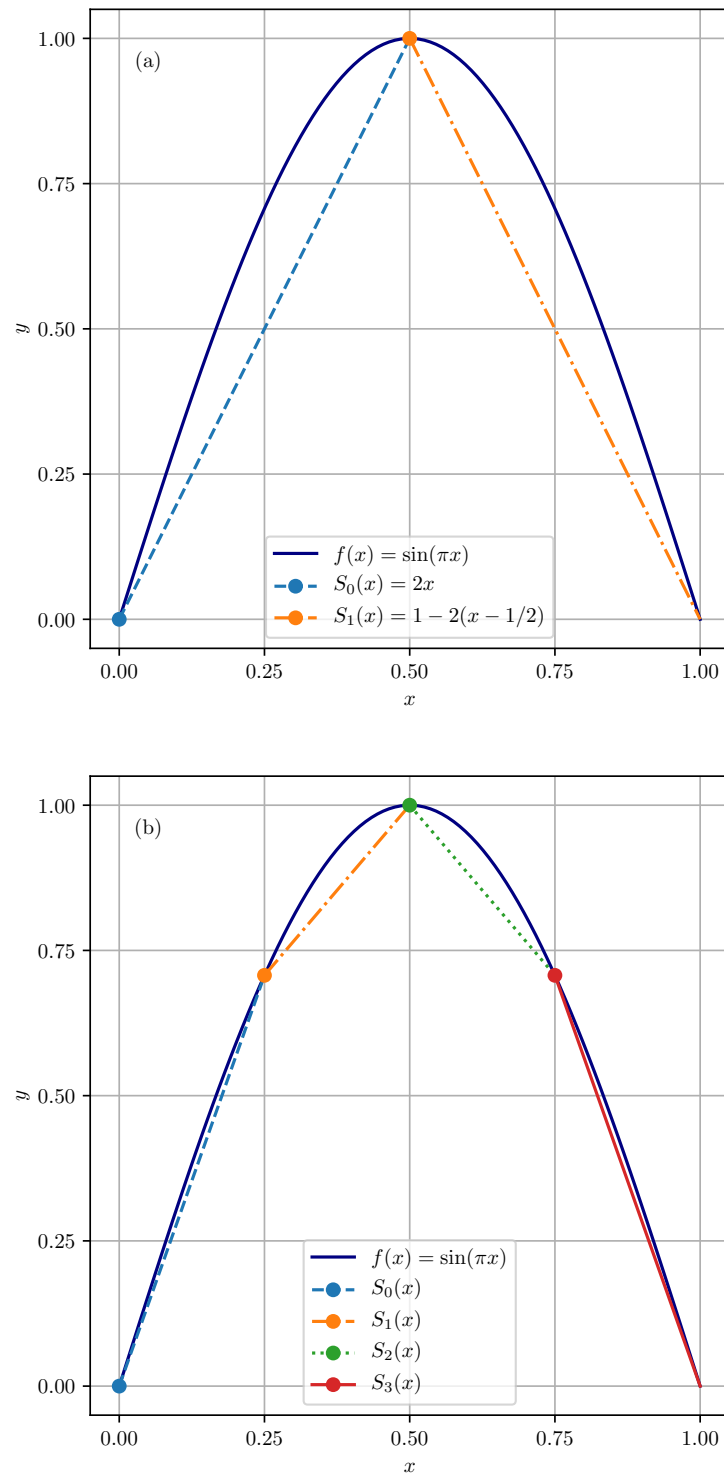


Figure 2.4: Linear spline example (a) with three points and (b) with five points, see Table 2.4 for individual definitions of $S_i(x)$. Reference: fig/linear_spline.py.

Bibliography

- [Bartels 1995] Bartels RH, Beatty JC, Barsky BA. An introduction to splines for use in computer graphics and geometric modeling. Morgan Kaufmann; 1995 Sep 15.
- [Rogers 2000] Rogers DF. An introduction to NURBS: with historical perspective. Elsevier; 2000 Aug 22.
- [Shiach 2015] Shiach J. Bézier Curves. <https://youtu.be/2HvH9cmHbG4>; 2015 Mar 09.